

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO ĐỒ ÁN VDT

NỘI DUNG

**Xây dựng hệ thống thu thập, lưu trữ, phân tích sử dụng
kiến trúc Data Lakehouse tích hợp theo dõi thị trường
chứng khoán**

Mentor hướng dẫn: Anh Bùi Lê Huy

1. Đặng Quốc Cường

MSSV: 23520192

TP. Hồ Chí Minh, 2026

Mục lục

1	Mục tiêu tuần 2	3
2	Bối cảnh bài toán	3
3	Mục tiêu tổng thể của hệ thống	3
4	Kiến trúc tổng thể	4
5	Tech stack dự kiến	5
6	Lý do sử dụng Polars thay cho Spark	6
7	Lý do sử dụng Apache Iceberg	6
8	Thiết kế batch pipeline chi tiết	6
8.1	Phân tách lịch chạy theo loại dữ liệu	7
9	Staging Layer	7
10	Thiết kế Lakehouse theo Medallion Architecture	8
10.1	Bronze Layer	8
10.2	Silver Layer	8
10.3	Gold Layer	8
11	Partition Strategy	9
12	Tính lũy đẳng của pipeline	9
13	Data Quality với Great Expectations	9
13.1	Kiểm tra Bronze	10
13.2	Kiểm tra Silver	10
13.3	Kiểm tra Gold	10
14	Thiết kế Star Schema cho tầng Gold	11
15	Bảng dim_symbol	11
16	Bảng dim_date	12
17	Bảng fact_hose_daily_market	13
18	Tính toán chỉ báo bằng Polars	13
19	Load dữ liệu sang ClickHouse	13
20	Dashboard batch bằng Streamlit	14
21	Các bảng realtime serving	14

22 Kết hợp batch và streaming	14
23 Bảo mật dữ liệu	14
24 Các dịch vụ Docker Compose	15
25 Kế hoạch triển khai giai đoạn 2	15
26 Kết luận tuần 2	15

1 Mục tiêu tuần 2

Mục tiêu của tuần 2 là hoàn thiện thiết kế batch pipeline và chuẩn bị triển khai thực tế các thành phần chính của hệ thống. Phạm vi dự án được giới hạn ở dữ liệu chứng khoán thuộc sàn HOSE nhằm giảm độ phức tạp trong quá trình thu thập dữ liệu, chuẩn hóa schema và xây dựng dashboard.

Các nội dung chính cần hoàn thành trong tuần 1 gồm:

- Xác định phạm vi dữ liệu thuộc sàn HOSE.
- Thiết kế kiến trúc kết hợp batch và streaming.
- Lựa chọn tech stack phù hợp với quy mô MVP.
- Thiết kế Medallion Architecture gồm Bronze, Silver và Gold.
- Thiết kế Star Schema cho tầng Gold của batch pipeline.
- Xác định grain của bảng fact chính.
- Tách rõ Lakehouse batch và các bảng realtime serving.
- Xác định cơ chế kiểm tra chất lượng dữ liệu.
- Bảo đảm pipeline có tính lũy đẳng khi chạy lại.
- Định hướng bảo mật bằng RBAC, RLS, masking và audit log.

2 Bối cảnh bài toán

Dữ liệu chứng khoán HOSE bao gồm dữ liệu OHLCV cuối ngày, thông tin doanh nghiệp niêm yết, dữ liệu giao dịch trong phiên, giá khớp gần thời gian thực, khối lượng giao dịch và các chỉ báo kỹ thuật như SMA, EMA, RSI, MACD và VWAP.

Các loại dữ liệu này thường được cung cấp từ nhiều nguồn khác nhau, không đồng nhất về schema và khó kết hợp giữa dữ liệu lịch sử với dữ liệu realtime. Vì vậy, dự án hướng tới xây dựng một hệ thống Data Lakehouse giúp thu thập, lưu trữ, làm sạch, kiểm soát chất lượng và phục vụ phân tích dữ liệu chứng khoán HOSE theo một kiến trúc thống nhất.

Trong phạm vi MVP, dự án ưu tiên triển khai batch pipeline hoàn chỉnh trước. Streaming pipeline được giữ trong kiến trúc tổng thể và sẽ được triển khai sau khi luồng batch hoạt động ổn định.

3 Mục tiêu tổng thể của hệ thống

Các mục tiêu chính gồm:

- Thu thập dữ liệu OHLCV lịch sử của các mã cổ phiếu HOSE.
- Thu thập metadata của doanh nghiệp niêm yết.

- Sinh calendar và dữ liệu ngày lễ để xây dựng `dim_date`.
- Lưu trữ dữ liệu batch trên MinIO theo định dạng bảng Apache Iceberg.
- Tổ chức dữ liệu theo Bronze, Silver và Gold.
- Kiểm tra chất lượng dữ liệu bằng Great Expectations.
- Xử lý và tính toán dữ liệu bằng Polars.
- Mô hình hóa dữ liệu Gold theo Star Schema.
- Tính các chỉ báo SMA20, EMA20, RSI14, MACD và volume trung bình.
- Load dữ liệu Gold sang ClickHouse để phục vụ truy vấn.
- Xây dựng dashboard bằng Streamlit và Plotly.
- Bảo đảm pipeline có thể chạy lại mà không tạo dữ liệu trùng lặp.
- Hỗ trợ snapshot, time travel và rollback bằng Apache Iceberg.

4 Kiến trúc tổng thể

Batch Pipeline:

VNStock API / VCI + Python holidays

- > Apache Airflow
- > Python + Polars
- > Staging Parquet trên MinIO
- > Great Expectations
- > PyIceberg
- > Bronze / Silver / Gold Iceberg trên MinIO
- > ClickHouse
- > Streamlit + Plotly

Streaming Pipeline:

DNSE API / Market Data API

- > Kafka Producer
- > Apache Kafka
- > ClickHouse Kafka Engine
- > Materialized View
- > Realtime Tables
- > Grafana

Batch + Streaming Integration:

- fact_hose_daily_market
- + rt_hose_latest_price
- + rt_hose_intraday_vwap
- > realtime_hose_stock_signal

-> hose_alert_events

MinIO đóng vai trò object storage. Apache Iceberg quản lý dữ liệu trên MinIO dưới dạng bảng. Polars đảm nhiệm xử lý dữ liệu in-memory, trong khi PyIceberg đảm nhiệm việc ghi, đọc và commit dữ liệu vào bảng Iceberg. ClickHouse đóng vai trò serving database để phục vụ truy vấn nhanh. Streamlit được sử dụng cho dashboard batch, trong khi Grafana được định hướng sử dụng cho dashboard realtime.

Riêng `dim_date` được sinh trực tiếp từ calendar 2024–2026 và dữ liệu ngày lễ của thư viện Python holidays. Bảng này không đi qua Bronze và Silver vì đây là dimension tham chiếu được tạo nội bộ, không phải dữ liệu nghiệp vụ thô lấy từ API.

5 Tech stack dự kiến

Thành phần	Công nghệ	Vai trò
Data source	VNStock API, VCI	Thu thập dữ liệu OHLCV và meta-data cổ phiếu
Calendar source	Python holidays	Sinh dữ liệu ngày lễ và ngày nghỉ bù tại Việt Nam
Orchestration	Apache Airflow	Lập lịch, retry và điều phối batch pipeline
Processing	Polars	Làm sạch, transform và tính các chỉ báo kỹ thuật
Data Quality	Great Expectations	Kiểm tra chất lượng dữ liệu trước khi ghi sang layer tiếp theo
Object Storage	MinIO	Lưu staging, data files và Iceberg metadata
Table format	Apache Iceberg	Quản lý bảng Bronze, Silver và Gold; hỗ trợ ACID, snapshot và time travel
Iceberg client	PyIceberg	Tạo bảng, append, overwrite và đọc dữ liệu Iceberg
Catalog	Iceberg REST Catalog	Quản lý metadata và commit của các bảng Iceberg
Message broker	Apache Kafka	Truyền dữ liệu realtime
Serving Database	ClickHouse	Phục vụ truy vấn nhanh cho batch và xử lý dữ liệu realtime
Batch Dashboard	Streamlit, Plotly	Trực quan hóa dữ liệu lịch sử, OHLCV và chỉ báo kỹ thuật
Realtime Dashboard	Grafana	Trực quan hóa dữ liệu realtime và cảnh báo
Deployment	Docker Compose	Triển khai và quản lý các dịch vụ trong môi trường local
Security	RBAC, RLS, Audit Log	Kiểm soát truy cập và truy vết hoạt động hệ thống

Bảng 1: Tech stack dự kiến

6 Lý do sử dụng Polars thay cho Spark

Trong phạm vi MVP, khối lượng dữ liệu batch còn nhỏ, chỉ gồm một số mã cổ phiếu HOSE và dữ liệu lịch sử trong khoảng thời gian giới hạn. Vì vậy, việc triển khai một Spark cluster có thể làm tăng độ phức tạp vận hành nhưng chưa mang lại nhiều lợi ích thực tế.

Polars được lựa chọn vì có tốc độ xử lý tốt, hỗ trợ lazy execution, tối ưu cho dữ liệu dạng cột và làm việc hiệu quả với Arrow và Parquet.

Trong kiến trúc này:

- Polars đảm nhiệm xử lý DataFrame.
- Great Expectations đảm nhiệm kiểm tra chất lượng.
- PyIceberg đảm nhiệm commit dữ liệu vào bảng Iceberg.
- Apache Iceberg đảm nhiệm snapshot, partition, time travel và ACID.

Spark có thể được bổ sung trong tương lai khi hệ thống mở rộng sang toàn bộ mã HOSE, dữ liệu nhiều năm hoặc dữ liệu intraday có khối lượng lớn.

7 Lý do sử dụng Apache Iceberg

Nếu chỉ lưu file Parquet trực tiếp trên MinIO, hệ thống chủ yếu mới ở mức Data Lake. Việc quản lý schema, version dữ liệu, partition và rollback sẽ trở nên khó khăn khi số lượng file tăng lên.

Apache Iceberg bổ sung lớp quản lý bảng trên object storage và hỗ trợ schema evolution, partition evolution, snapshot, time travel, rollback, transaction ở mức bảng, atomic commit và optimistic concurrency control.

Các tính năng trên không đến từ Polars. Polars chỉ xử lý dữ liệu. Dữ liệu sau khi xử lý được chuyển thành Arrow Table và ghi vào Iceberg thông qua PyIceberg.

```
Polars DataFrame  
-> Arrow Table  
-> PyIceberg  
-> Iceberg REST Catalog  
-> MinIO
```

8 Thiết kế batch pipeline chi tiết

Batch pipeline được thiết kế theo các bước sau:

1. Airflow khởi tạo DAG theo lịch.
2. Python gọi VNStock API để lấy OHLCV và metadata symbol.

3. Dữ liệu API được chuyển thành Polars DataFrame.
4. Polars ghi dữ liệu tạm dưới dạng Parquet vào vùng staging trên MinIO.
5. Great Expectations kiểm tra schema và các rule chất lượng cơ bản.
6. PyIceberg ghi dữ liệu OHLCV và symbol hợp lệ vào Bronze Iceberg.
7. Polars đọc Bronze, làm sạch và chuẩn hóa dữ liệu.
8. Great Expectations kiểm tra dữ liệu Silver.
9. PyIceberg ghi dữ liệu OHLCV và symbol vào Silver Iceberg.
10. Python sinh calendar 2024–2026 và dữ liệu ngày lễ, sau đó tạo trực tiếp `dim_date`.
11. Polars tạo `dim_symbol` từ metadata symbol đã chuẩn hóa.
12. Polars tính chỉ báo và xây dựng `fact_hose_daily_market`.
13. Great Expectations kiểm tra khóa, duplicate và business rules ở Gold.
14. PyIceberg ghi các bảng Gold.
15. Dữ liệu Gold được load vào ClickHouse.
16. Streamlit đọc dữ liệu từ ClickHouse để hiển thị dashboard.

8.1 Phân tách lịch chạy theo loại dữ liệu

Batch pipeline được chia thành các nhóm job có tần suất khác nhau:

- `dim_date`: tạo sẵn một lần cho giai đoạn 2024–2026 và chỉ mở rộng khi cần thêm năm mới.
- `dim_symbol`: bootstrap từ toàn bộ danh sách mã HOSE, sau đó refresh định kỳ hằng tuần hoặc hằng tháng.
- `fact_hose_daily_market`: chạy hằng ngày theo dữ liệu OHLCV của ngày D.

Fact chỉ được ghi sau khi hai dimension đã tồn tại. Trong quá trình tạo fact, `symbol` được ánh xạ sang `symbol_key` và `trading_date` được ánh xạ sang `date_key`.

9 Staging Layer

Staging là vùng trung gian giữa quá trình gọi API và quá trình commit vào Iceberg.

```
s3://lakehouse/staging/ohlcw/
s3://lakehouse/staging/symbols/
```

Staging có các mục đích:

- Giữ dữ liệu ngay sau khi lấy từ nguồn.
- Cho phép retry mà không cần gọi lại API.
- Hỗ trợ kiểm tra và debug.
- Tách bước extract khỏi bước commit Iceberg.
- Lưu lại input của từng batch.

10 Thiết kế Lakehouse theo Medallion Architecture

10.1 Bronze Layer

Bronze lưu dữ liệu gần với nguồn nhất và chỉ bổ sung metadata phục vụ quản lý pipeline.

- `bronze_hose_ohlcv_daily`
- `bronze_hose_symbols`

Các cột metadata được bổ sung gồm `source`, `batch_id` và `ingested_at`.

10.2 Silver Layer

Silver lưu dữ liệu đã được làm sạch và chuẩn hóa.

- `silver_hose_ohlcv_daily`
- `silver_hose_symbols`

Các xử lý chính gồm chuẩn hóa tên cột, ép kiểu dữ liệu, chuẩn hóa mã cổ phiếu, lọc đúng khoảng thời gian, loại duplicate theo `symbol + trading_date`, kiểm tra tính hợp lệ của OHLCV và chuẩn hóa metadata doanh nghiệp.

10.3 Gold Layer

Gold chứa dữ liệu đã được mô hình hóa theo Star Schema để phục vụ dashboard và truy vấn phân tích.

- `dim_symbol`
- `dim_date`
- `fact_hose_daily_market`

Trong phạm vi HOSE-only, không tạo riêng `dim_exchange` vì toàn bộ dữ liệu đều thuộc HOSE. Trường `exchange_code` được lưu trực tiếp trong `dim_symbol`. Thông tin ngành cũng được lưu trong `dim_symbol` thông qua cột `sector_name`.

11 Partition Strategy

Các bảng OHLCV và bảng fact được partition theo ngày. Partition được quản lý bởi Apache Iceberg, không phải bởi Polars.

Bảng	Partition	Ghi chú
bronze_hose_ohlcvc_daily	day(trading_date)	Phục vụ ingest và retry theo ngày
silver_hose_ohlcvc_daily	day(trading_date)	Phục vụ transform theo ngày
fact_hose_daily_market	day(trading_date)	Phục vụ overwrite và query theo ngày
dim_symbol	Không partition	Bảng nhỏ
dim_date	Không partition	Bảng nhỏ

Bảng 2: Partition strategy cho các bảng batch

12 Tính lũy đẳng của pipeline

Pipeline cần bảo đảm rằng khi chạy lại nhiều lần với cùng một input, trạng thái dữ liệu cuối cùng không thay đổi và không xuất hiện duplicate.

Lấy đầy đủ dữ liệu ngày D

-> Polars transform

-> Great Expectations validate

-> PyIceberg dynamic partition overwrite

Cơ chế dynamic partition overwrite thay thế toàn bộ dữ liệu thuộc partition ngày D bằng batch mới, nhưng không làm ảnh hưởng đến các ngày khác. Nhờ đó, Airflow retry không tạo duplicate, backfill một ngày không làm tăng số dòng và dữ liệu điều chỉnh có thể được ghi lại đúng partition. Mỗi lần ghi vẫn tạo snapshot Iceberg mới nên có thể time travel về dữ liệu trước khi overwrite.

Trước khi overwrite, batch phải chứa đầy đủ dữ liệu của ngày D. Nếu batch thiếu mã cổ phiếu, dữ liệu cũ của các mã bị thiếu sẽ không còn xuất hiện trong snapshot mới.

13 Data Quality với Great Expectations

Great Expectations được sử dụng để kiểm tra dữ liệu trước mỗi lần commit vào Iceberg. Do dữ liệu được xử lý bằng Polars, dữ liệu có thể được chuyển sang Pandas hoặc Arrow để chạy Great Expectations trong phạm vi từng batch.

Polars DataFrame

-> Pandas DataFrame

-> Great Expectations

-> PyIceberg commit

13.1 Kiểm tra Bronze

- `symbol` không null.
- `time` không null.
- Các cột OHLCV có đúng kiểu dữ liệu.
- `source` không null.
- `batch_id` không null.

13.2 Kiểm tra Silver

- `symbol` không null.
- `trading_date` không null.
- Không duplicate theo `symbol` + `trading_date`.
- `open_price` > 0.
- `high_price` >= `low_price`.
- `high_price` >= `open_price`.
- `high_price` >= `close_price`.
- `low_price` <= `open_price`.
- `low_price` <= `close_price`.
- `volume` >= 0.
- Batch chỉ chứa ngày đang được xử lý.

13.3 Kiểm tra Gold

- `symbol_key` không null.
- `date_key` không null.
- Không duplicate theo `symbol_key` + `date_key`.
- `rsi14` nằm trong khoảng từ 0 đến 100.
- Các foreign key phải tồn tại trong dimension.
- Số lượng symbol trong batch đạt mức kỳ vọng.

Nếu validation thất bại, task sẽ dừng trước khi commit. Dữ liệu lỗi được lưu vào vùng quarantine để phục vụ kiểm tra.

14 Thiết kế Star Schema cho tầng Gold

```

        dim_symbol
          |
          |
dim_date -- fact_hose_daily_market

```

Bảng `fact_hose_daily_market` là bảng trung tâm. Hai bảng `dim_symbol` và `dim_date` được sử dụng để lọc, nhóm và phân tích dữ liệu.

15 Bảng `dim_symbol`

Grain của `dim_symbol` là một dòng cho một mã cổ phiếu.

Khác với bảng `fact` chỉ mới thử nghiệm trên một số mã cổ phiếu, bảng `dim_symbol` được khởi tạo trước từ toàn bộ danh sách mã đang niêm yết trên sàn HOSE. Việc này giúp xây dựng sẵn bảng tra cứu dùng chung cho toàn bộ hệ thống và tránh phải thiết kế lại dimension khi mở rộng phạm vi dữ liệu.

Luồng xử lý metadata symbol được thiết kế như sau:

```

VNStock Listing / Company API
-> lấy toàn bộ symbol thuộc HOSE
-> Polars chuẩn hóa
-> Great Expectations validate
-> bronze_hose_symbols
-> silver_hose_symbols
-> dim_symbol
-> ClickHouse

```

Bảng `dim_symbol` được bootstrap một lần trước khi load fact. Sau đó, metadata được refresh định kỳ bằng một Airflow DAG riêng.

```

dag_symbol_metadata
|
+--> extract_hose_symbols
+--> write_bronze_symbols
+--> validate_bronze_symbols
+--> transform_silver_symbols
+--> validate_silver_symbols
+--> upsert_dim_symbol
+--> sync_dim_symbol_to_clickhouse

```

Lịch chạy phù hợp là hằng tuần hoặc hằng tháng vì metadata doanh nghiệp thay đổi chậm hơn dữ liệu OHLCV daily.

Khi refresh:

- Symbol đã tồn tại được cập nhật metadata nhưng giữ nguyên `symbol_key`.
- Symbol mới niêm yết được thêm vào và cấp `symbol_key` mới.
- Symbol hủy niêm yết được cập nhật `listed_status`, không xóa cứng.
- Không append mù để tránh duplicate.

Cột	Kiểu	Mô tả
<code>symbol_key</code>	UInt32	Surrogate key ổn định của mã cổ phiếu
<code>symbol</code>	String	Mã cổ phiếu
<code>company_name</code>	String	Tên doanh nghiệp
<code>sector_name</code>	String	Nhóm ngành của doanh nghiệp
<code>company_profile</code>	String	Mô tả doanh nghiệp
<code>listing_date</code>	Date	Ngày niêm yết
<code>issued_share</code>	UInt64	Số lượng cổ phiếu phát hành nếu có
<code>exchange_code</code>	String	Giá trị cố định HOSE
<code>listed_status</code>	String	Trạng thái niêm yết
<code>updated_at</code>	DateTime	Thời điểm cập nhật metadata gần nhất

Bảng 3: Schema bảng `dim_symbol`

Trong giai đoạn MVP, `dim_symbol` có thể chứa toàn bộ mã HOSE trong khi bảng `fact_hose_daily_market` chỉ mới chứa dữ liệu của một số mã dùng để kiểm thử pipeline.

16 Bảng `dim_date`

Grain của `dim_date` là một dòng cho một ngày lịch. Bảng được sinh trực tiếp ở tầng Gold cho phạm vi từ năm 2024 đến năm 2026. Dữ liệu ngày lễ Việt Nam được tạo bằng thư viện Python `holidays` và merge vào `calendar` đầy đủ. Không tạo các bảng `bronze_calendar` hoặc `silver_calendar_events`.

Cột	Kiểu	Mô tả
<code>date_key</code>	UInt32	Surrogate key của ngày
<code>full_date</code>	Date	Ngày đầy đủ
<code>day</code>	UInt8	Ngày trong tháng
<code>cal_week</code>	UInt8	Tuần trong năm
<code>cal_month</code>	UInt8	Tháng
<code>cal_quarter</code>	UInt8	Quý
<code>cal_year</code>	UInt16	Năm
<code>is_weekend</code>	UInt8	Đánh dấu ngày cuối tuần
<code>event_name</code>	Nullable(String)	Tên ngày lễ hoặc sự kiện
<code>event_type</code>	Nullable(String)	Holiday hoặc Compensation
<code>is_day_off</code>	UInt8	Ngày cuối tuần hoặc ngày lễ

Bảng 4: Schema bảng `dim_date`

Cột `is_day_off` không được gọi là `is_trading_day`, do dữ liệu ngày lễ không phải lịch giao dịch chính thức của HOSE.

17 Bảng `fact_hose_daily_market`

Grain của bảng `fact` là một mã cổ phiếu trong một ngày giao dịch. Bảng `fact` gộp dữ liệu OHLCV daily và các chỉ báo kỹ thuật vì chúng có cùng grain.

Cột	Kiểu	Mô tả
<code>symbol_key</code>	UInt32	Khóa liên kết tới <code>dim_symbol</code>
<code>date_key</code>	UInt32	Khóa liên kết tới <code>dim_date</code>
<code>trading_date</code>	Date	Ngày giao dịch, dùng cho partition
<code>open_price</code>	Float64	Giá mở cửa
<code>high_price</code>	Float64	Giá cao nhất
<code>low_price</code>	Float64	Giá thấp nhất
<code>close_price</code>	Float64	Giá đóng cửa
<code>volume</code>	UInt64	Khối lượng giao dịch
<code>price_change</code>	Float64	Thay đổi giá so với phiên trước
<code>pct_change</code>	Float64	Phần trăm thay đổi giá
<code>sma20</code>	Nullable(Float64)	SMA 20 phiên
<code>ema20</code>	Nullable(Float64)	EMA 20 phiên
<code>rsi14</code>	Nullable(Float64)	RSI 14 phiên
<code>macd</code>	Nullable(Float64)	MACD
<code>avg_volume_20d</code>	Nullable(Float64)	Volume trung bình 20 phiên
<code>updated_at</code>	DateTime	Thời điểm cập nhật

Bảng 5: Schema bảng `fact_hose_daily_market`

18 Tính toán chỉ báo bằng Polars

Polars được sử dụng để tính `price_change`, `pct_change`, `sma20`, `ema20`, `rsi14`, `macd` và `avg_volume_20d`. Các phép tính được thực hiện theo từng mã cổ phiếu và sắp xếp theo ngày giao dịch.

19 Load dữ liệu sang ClickHouse

Sau khi Gold Iceberg được tạo và kiểm tra thành công, Airflow thực hiện load dữ liệu vào ClickHouse.

```
Gold Iceberg
-> PyIceberg scan
-> Polars
-> clickhouse-connect
-> ClickHouse
```

Các bảng serving gồm `dim_symbol`, `dim_date` và `fact_hose_daily_market`. Quá trình load ClickHouse cũng phải bảo đảm tính lũy đẳng. Dữ liệu của ngày đang xử lý được xóa hoặc thay thế trước khi insert lại, tránh duplicate khi Airflow retry.

20 Dashboard batch bằng Streamlit

Streamlit kết nối trực tiếp tới ClickHouse để hiển thị dashboard lịch sử. Các thành phần dashboard dự kiến gồm biểu đồ nến OHLCV, SMA20, EMA20, RSI14, MACD, volume, volume trung bình 20 phiên, thông tin doanh nghiệp và bảng dữ liệu OHLCV. Plotly được sử dụng để xây dựng biểu đồ nến và các biểu đồ tương tác.

21 Các bảng realtime serving

Các bảng realtime không thuộc Gold Star Schema. Chúng được lưu trong ClickHouse để phục vụ dashboard và alert gần thời gian thực.

- `rt_hose_stock_ticks`
- `rt_hose_latest_price`
- `rt_hose_ohlc_v1m`
- `rt_hose_intraday_vwap`
- `realtime_hose_stock_signal`
- `hose_alert_events`

22 Kết hợp batch và streaming

Dữ liệu Gold batch cung cấp bối cảnh lịch sử như SMA20, EMA20, RSI14, MACD và volume trung bình. Dữ liệu streaming cung cấp latest price, volume realtime và VWAP trong phiên.

```
fact_hose_daily_market
+ rt_hose_latest_price
+ rt_hose_intraday_vwap
-> realtime_hose_stock_signal
-> hose_alert_events
```

23 Bảo mật dữ liệu

Các cơ chế bảo mật dự kiến gồm:

- **RBAC**: phân quyền admin, data engineer, analyst và viewer.
- **RLS**: giới hạn dữ liệu theo mã hoặc nhóm ngành.
- **Column-Level Security**: giới hạn quyền truy cập cột nhạy cảm.
- **Data Masking**: che API key, token và thông tin nhạy cảm.
- **Audit Log**: ghi lịch sử truy cập, query và thay đổi quyền.

24 Các dịch vụ Docker Compose

Các dịch vụ cần thiết cho batch MVP gồm Airflow Webserver, Airflow Scheduler, PostgreSQL cho Airflow, MinIO, MinIO initialization service, Iceberg REST Catalog, backend metadata cho catalog, ClickHouse và Streamlit.

Spark không được triển khai trong MVP vì Polars và PyIceberg đã đáp ứng quy mô dữ liệu hiện tại.

25 Kế hoạch triển khai giai đoạn 2

1. Dựng MinIO bằng Docker Compose.
2. Dựng Iceberg REST Catalog.
3. Kiểm tra PyIceberg tạo và ghi một bảng Iceberg.
4. Xây dựng module lấy dữ liệu bằng VNStock.
5. Tạo staging Parquet trên MinIO.
6. Tích hợp Great Expectations.
7. Tạo các bảng Bronze Iceberg.
8. Xây dựng transform Bronze sang Silver bằng Polars.
9. Tạo `dim_date` và `dim_symbol`.
10. Tính chỉ báo và tạo bảng fact Gold.
11. Cài đặt dynamic partition overwrite theo ngày.
12. Kiểm thử tính lũy đẳng của pipeline.
13. Load Gold sang ClickHouse.
14. Xây dựng dashboard Streamlit.
15. Đưa toàn bộ luồng vào Airflow DAG.

26 Kết luận tuần 2

Trong tuần 2, dự án đã hoàn thiện thiết kế batch pipeline cho dữ liệu HOSE và xác định rõ cách tổ chức các job Airflow theo từng loại dữ liệu.

Batch pipeline sử dụng Airflow để điều phối, Polars để xử lý dữ liệu, Great Expectations để kiểm tra chất lượng, PyIceberg để commit bảng Iceberg và MinIO để lưu trữ dữ liệu.

Các bảng OHLCV và bảng fact được partition theo ngày. Pipeline sử dụng dynamic partition overwrite để thay thế dữ liệu của ngày đang xử lý, qua đó bảo đảm tính lũy đẳng khi Airflow retry hoặc backfill.

Gold Layer được rút gọn thành ba bảng chính gồm `dim_symbol`, `dim_date` và `fact_hose_daily_market`. Các bảng realtime được tách riêng khỏi Star Schema và lưu trong ClickHouse.

ClickHouse đóng vai trò serving database, trong khi Streamlit và Plotly được sử dụng để xây dựng dashboard batch. Spark chưa được sử dụng trong MVP nhưng có thể được bổ sung khi khối lượng dữ liệu tăng và cần xử lý phân tán.